算法基础：习题解答 2

2025 春　任课老师：陈雪　　due: March 17, 23:59

作业要求：说明思路与符号，清晰简洁的伪代码，必要的时间复杂度分析和必要的正确性分析。可以直接调用基本的数据结构和已讨论过的算法 / 程序（如排序、找中位数、二分查找等）。

问题 1（20 分）

(a)　令 $T(n)$ 为 n 个元素的快速排序期望时间。对课件中的 QuickSort 代码，写出 $T(n)$ 的递归式。参考《算法导论》练习题 7-3。

(b)　证明递归式 $T(n) = O(n \log n)$。

Solution.

(a)　当 $n \geq 2$ 时，由于 pivot 随机选取，有

$$\begin{aligned} T(n) &= \frac{1}{n} \sum_{i=1}^{n} \big(T(i-1) + T(n-i)\big) + \Theta(n) \\ &= \frac{2}{n} \sum_{i=1}^{n} T(i-1) + \Theta(n). \end{aligned}$$

这里使用了 $\sum_{i=1}^{n} T(i-1) = \sum_{i=1}^{n} T(n-i)$。边界为 $T(0), T(1) = \Theta(1)$。

(b)　归一化后取 $T(0) = T(1) = 1$。选取足够大的常数 C，使递推式中的线性开销不超过 Cn，并使 $C \geq 3$、基例 $T(2) \leq 4C \log 2$ 成立。下面证明

$$T(n) \leq 2Cn \log n, \qquad n \geq 2,$$

这里的对数为自然对数。

假设对所有 $2 \leq i < n$ 均有 $T(i) \leq 2Ci \log i$，考虑 n：

$$\begin{aligned} \sum_{i=1}^{n} T(i-1) &= T(0) + T(1) + \sum_{i=2}^{n-1} T(i) \\ &\leq 2 + 2C \sum_{i=2}^{n-1} i \log i \\ &\leq 2 + 2C \int_{2}^{n} x \log x \, dx \qquad (1) \\ &= 2 + 2C \left[\frac{x^2}{2} \log x - \frac{x^2}{4} \right]_2^n \\ &= 2 + 2C \left(\frac{n^2}{2} \log n - \frac{n^2}{4} - 2 \log 2 + 1 \right) \\ &= Cn^2 \log n - \frac{Cn^2}{2} + (2 - 4 \log 2)C + 2 \\ &\leq Cn^2 \log n - \frac{Cn^2}{2}. \end{aligned}$$

式 (1) 使用了 $f(x) = x \log x$ 在 $[2, +\infty)$ 上单调递增。最后一步由 $C \geq 3$ 得到。

使用以上不等式，

$$T(n) \leq \frac{2}{n} \sum_{i=1}^{n} T(i-1) + Cn$$

$$\leq \frac{2}{n}\left(Cn^2\log n - \frac{Cn^2}{2}\right) + Cn$$
$$= 2Cn\log n - Cn + Cn = 2Cn\log n.$$

由数学归纳法，$T(n) = O(n\log n)$。

问题 2（20 分）

(a) 基于课件的 QuickSort 代码，设计期望时间为 $O(n)$ 的寻找第 k 号数的算法（从小到大）。

(b) 计算每对元素的比较概率，并证明算法的期望时间。

(c) k 取何值时，（b）中每对元素比较概率之和最大？

Solution.

(a) 初始调用为 QuickFind$(1, n, k)$，其中 k 是整个数组中从小到大的目标下标。

Algorithm 1 QuickFind

```
1: procedure QuickFind(p, r, k)
2:     if p = r then
3:         return A_p
4:     end if
5:     q ← Partition(p, r)
6:     if q = k then
7:         return A_q
8:     else if q > k then
9:         return QuickFind(p, q − 1, k)
10:    else
11:        return QuickFind(q + 1, r, k)
12:    end if
13: end procedure
```

其中 Partition 函数与课件中一致，随机选取 pivot。

记 $z_1 < z_2 < \cdots < z_n$ 为原数组排序后的结果。特别地，定义

$$A_0 = z_0 = -\infty, \qquad A_{n+1} = z_{n+1} = +\infty.$$

Lemma 1.

每次 Partition 之后，有 $A_q = z_q$。

证明. 对于课件中的 QuickSort，每次确定 q 之后，A_q 将不再变化。算法结束时数组排好序，因此 $A_q = z_q$。本算法的递归过程是 QuickSort 的一种特殊情况：只递归到包含目标下标的分支，同样有 $A_q = z_q$。 □

根据该引理，若在 $q = k$ 时返回，则必有 $A_q = z_k$，算法正确。若在 $p = r$ 时返回，则：

- 当前左端点 p 必然来自 1 或之前某个 pivot 下标 $q' + 1$，因此 $A_{p-1} = z_{p-1}$；同理 $A_{p+1} = z_{p+1}$。
- 由划分过程的性质，$A_{p-1} < A_p < A_{p+1}$。
- 只剩 A_p 这个元素，其全局秩必为 p，即 $A_p = z_p$；当前区间始终包含 k，故 $p = r = k$。

综上，证明了算法的正确性。

(b) 设 $z_1 < z_2 < \cdots < z_n$ 为 A 排序后的结果。分析任意两个元素 z_i, z_j（$i < j$）会被比较的概率。

当 $i < j < k$ 时，考虑区间 $[i, k]$ 中最先成为 pivot 的元素：

- 若 z_i 最先成为 pivot，则 z_i 会与 z_j 比较。
- 若 $x \in (i, j)$，z_x 最先成为 pivot，则 z_i, z_j 分散在两个分支中，不会比较。

- 若 z_j 最先成为 pivot，则 z_j 会与 z_i 比较。
- 若 $x \in (j,k)$，z_x 最先成为 pivot，则包含 z_i, z_j 的分支不会继续递归，因此不会比较。
- 若 z_k 最先成为 pivot，算法直接结束，z_i, z_j 不会比较。

因此，该概率为 $2/(k-i+1)$。对于 $i<k<j$ 和 $k<i<j$，进行类似分析。与 z_k 配对的情形也包含在内，完整结果为

$$\mathbb{P}(z_i, z_j \text{ 被比较}) = \begin{cases} \dfrac{2}{k-i+1}, & i<j\le k, \\ \dfrac{2}{j-i+1}, & i<k<j, \\ \dfrac{2}{j-k+1}, & k\le i<j. \end{cases}$$

等价地，统一写作

$$\mathbb{P}(z_i, z_j \text{ 被比较}) = \frac{2}{\max(j,k)-\min(i,k)+1}.$$

每对元素最多比较一次，故期望比较次数为

$$\begin{aligned} E(n,k) &= \sum_{1\le i<j\le k} \frac{2}{k-i+1} + \sum_{1\le i<k<j\le n} \frac{2}{j-i+1} + \sum_{k\le i<j\le n} \frac{2}{j-k+1} \\ &= E_1 + E_2 + E_3. \end{aligned}$$

下面分别估计三部分。

$$\begin{aligned} E_1 &= \sum_{1\le i<j\le k} \frac{2}{k-i+1} = 2\sum_{i=1}^{k-1}\sum_{j=i+1}^{k} \frac{1}{k-i+1} \\ &= 2\sum_{i=1}^{k-1} \frac{k-i}{k-i+1} \le 2\sum_{i=1}^{k-1} 1 \le 2k. \end{aligned}$$

对 E_3 作类似分析，有 $E_3 \le 2(n-k)$。接下来分析 E_2。令 $x = k-i$、$y = j-k+1$，则

$$\begin{aligned} E_2 &= 2\sum_{i=1}^{k-1}\sum_{j=k+1}^{n} \frac{1}{j-i+1} \\ &= 2\sum_{x=1}^{k-1}\sum_{y=2}^{n-k+1} \frac{1}{(y+k-1)-(k-x)+1} \\ &= 2\sum_{x=1}^{k-1}\sum_{y=2}^{n-k+1} \frac{1}{x+y} \\ &= 2\sum_{x=1}^{k-1}\sum_{y=x+2}^{n-k+1+x} \frac{1}{y} \\ &\le 2\sum_{x=1}^{k-1}\int_{x+1}^{n-k+1+x} \frac{1}{y}\,dy \qquad (2) \\ &= 2\sum_{x=1}^{k-1}\big(\ln(n-k+1+x) - \ln(1+x)\big) \\ &= 2\sum_{x=n-k+2}^{n} \ln x - 2\sum_{x=2}^{k} \ln x \\ &\le 2\int_{n-k+2}^{n+1} \ln x\,dx - 2\int_{1}^{k} \ln x\,dx \qquad (3) \\ &= 2(n+1)(\ln(n+1)-1) \\ &\quad - 2(n-k+2)(\ln(n-k+2)-1) - 2k(\ln k - 1) + 2(\ln 1 - 1) \end{aligned}$$

$$= 2(n+1)\ln(n+1) - 2(n-k+2)\ln(n-k+2) - 2k\ln k.$$

式 (2) 使用了 $1/y$ 在 $[1,+\infty)$ 上严格单调递减；式 (3) 使用了 $\ln x$ 在 $[1,+\infty)$ 上严格单调递增。边界 $k=1$ 或 $k=n$ 时，$E_2=0$，无需作积分估计。

由于 $f(x)=x\ln x$ 是凸函数，

$$(n-k+2)\ln(n-k+2) + k\ln k \geq (n+2)\ln\frac{n+2}{2}.$$

代入可得

$$\begin{aligned} E_2 &\leq 2(n+1)\ln(n+1) - 2(n+2)\ln\frac{n+2}{2} \\ &\leq 2(n+2)\ln(n+2) - 2(n+2)\ln\frac{n+2}{2} \\ &= 2(n+2)\ln 2 \leq 4(n+2). \end{aligned}$$

综上，

$$E(n,k) \leq 2k + 2(n-k) + 4(n+2) = O(n).$$

每次划分的其他开销与划分时的比较次数成正比，故算法期望时间为 $O(n)$。

(c) 该问题是对称的，$E(n,k) = E(n,n+1-k)$。只需考虑 k 处在前半段，并证明它向中间移动时，比较概率之和严格增加。

考虑 $\Delta E = E(n,k+1) - E(n,k)$。两种目标秩下各元素对贡献的差异分为两部分：

- 元素对中有一个下标是 k 或 $k+1$，这部分差值记为 ΔE_a。对 $(k,k+1)$，两种情况下概率均为 1，贡献相消。其余贡献为

$$\begin{aligned} \Delta E_a = &\sum_{i=1}^{k-1}\left(\frac{2}{k-i+2} - \frac{2}{k-i+1}\right) \\ &+ \sum_{j=k+2}^{n}\left(\frac{2}{j-k} - \frac{2}{j-k+1}\right). \end{aligned}$$

- 两个下标均不为 $k, k+1$，这部分差值记为 ΔE_b。跨过两个目标下标的元素对贡献不变；其余差值为

$$\begin{aligned} \Delta E_b = &\sum_{1\leq i<j\leq k-1}\left(\frac{2}{k-i+2} - \frac{2}{k-i+1}\right) \\ &+ \sum_{k+2\leq i<j\leq n}\left(\frac{2}{j-k} - \frac{2}{j-k+1}\right) \\ = &\sum_{i=1}^{k-2}(k-1-i)\left(\frac{2}{k-i+2} - \frac{2}{k-i+1}\right) \\ &+ \sum_{j=k+3}^{n}(j-k-2)\left(\frac{2}{j-k} - \frac{2}{j-k+1}\right). \end{aligned}$$

代入 $i = k-x$、$j = k+1+y$，得到

$$\begin{aligned} \Delta E_b = &\sum_{x=2}^{k-1}(x-1)\left(\frac{2}{x+2} - \frac{2}{x+1}\right) \\ &- \sum_{y=2}^{n-k-1}(y-1)\left(\frac{2}{y+2} - \frac{2}{y+1}\right). \end{aligned}$$

与 ΔE_a 合并后，

$$\Delta E = \sum_{x=1}^{k-1} x\left(\frac{2}{x+2} - \frac{2}{x+1}\right) - \sum_{y=1}^{n-k-1} y\left(\frac{2}{y+2} - \frac{2}{y+1}\right).$$

定义

$$f(s) = \sum_{x=1}^{s-1} x\left(\frac{2}{x+2} - \frac{2}{x+1}\right).$$

每个新增项均严格为负，因此 $f(s)$ 严格单调递减。于是

$$E(n,k+1) - E(n,k) = f(k) - f(n-k).$$

当 $n-k > k$，即 $k < n/2$ 时，上式严格为正；当 $n-k = k$ 时，上式为零。结合对称性，可得：

$$\begin{cases} n \text{ 为奇数：} & k = (n+1)/2 \text{ 时最大;} \\ n \text{ 为偶数：} & k = n/2 \text{ 或} n/2+1 \text{ 时最大.} \end{cases}$$

问题 3（20 分）

参考《算法导论》第 8.1 章，考虑 σ 为 $1,\ldots,n$ 的所有 $n!$ 个排列中均匀随机产生的排列。

(a) 对任何基于比较排序的确定性算法 A，证明至少 99% 的排列 σ 上，A 的时间为 $\Omega(n\log n)$，即对充分大的 n，

$$\mathbb{P}_\sigma\left[\text{Time}(A(\sigma)) \geq 0.5n\log n\right] \geq 0.99.$$

其中 $\text{Time}(A(\sigma))$ 表示算法 A 在输入 σ 上的运行时间。

(b) 考虑任意基于比较排序的随机化算法 $A(\sigma, r)$：永远输出正确答案，但运行时间为随机变量；其中 σ 为输入串，r 为随机种子。证明至少 90% 的排列 σ 满足

$$\mathbb{E}_r[\text{Time}(A(\sigma,r))] \geq 0.4n\log n.$$

提示：假定算法 A 采用固定长度 $l(n)$ 的随机数 r，即 $r \in \{0,1\}^{l(n)}$。固定随机数 r 后，$A(\cdot, r)$ 为确定性算法。

Solution.

(a) 使用反证法，假设

$$\mathbb{P}_\sigma[\text{Time}(A(\sigma)) \geq 0.5n\log n] < 0.99.$$

则

$$\mathbb{P}_\sigma[\text{Time}(A(\sigma)) < 0.5n\log n] > 0.01,$$

因此，快于该界的排列数超过 $0.01n!$。这些排列在比较决策树中必须落在不同的叶子上，且叶子的深度小于 $0.5n\log n$。二叉决策树在深度小于该界处最多容纳 $2^{0.5n\log n}$ 个叶子，从而

$$2^{0.5n\log n} \geq 0.01n!.$$

两边取以 2 为底的对数，得到

$$0.5n\log n + \log_2 100 \geq \log_2(n!).$$

下面给出一种不使用 Stirling 公式的方法。令 $q = \lfloor n/4 \rfloor$，当 n 足够大时，下列区间不相交，且 $q \geq 5$，所以

$$\log_2(n!) = \sum_{i=1}^{n} \log_2 i$$

$$\geq \sum_{i=1}^{5} \log_2 i + \sum_{i=q+1}^{2q} \log_2 i + \sum_{i=2q+1}^{4q} \log_2 i$$
$$> \log_2 100 + q \log_2 q + 2q \log_2 (2q).$$

这里第一段乘积为 $5! = 120 > 100$。当 n 是 4 的倍数时，以上就是

$$\log_2(n!) > \log_2 100 + \frac{n}{4} \log_2 \frac{n}{4} + \frac{n}{2} \log_2 \frac{n}{2}.$$

此时

$$\begin{aligned} \log_2(n!) - (0.5n \log_2 n + \log_2 100) &> \frac{n}{4} \log_2 \frac{n}{4} + \frac{n}{2} \log_2 \frac{n}{2} - \frac{n}{2} \log_2 n \\ &= \frac{n}{4} \log_2 \frac{n}{4} - \frac{n}{2} > 0 \end{aligned}$$

（充分大的 n）。对于任意 n，$q = \lfloor n/4 \rfloor$ 的取整只带来 $O(\log n)$ 误差；上述下界为

$$\log_2(n!) \geq \frac{3}{4} n \log_2 n - O(n),$$

仍严格大于 $0.5n \log_2 n + \log_2 100$。若 log 为自然对数，则 $\log n \leq \log_2 n$，所以也有

$$0.5n \log n + \log_2 100 \leq 0.5n \log_2 n + \log_2 100 < \log_2(n!).$$

这与假设矛盾，从而结论成立。

(b) 设指示变量

$$I(\sigma, r) = \mathbf{1}\{\mathrm{Time}(A(\sigma, r)) \geq 0.5n \log n\}.$$

这是一个 $0,1$ 变量，其概率与期望相等。对于固定的随机序列 r，由（a）得

$$\mathbb{E}_\sigma[I(\sigma, r)] \geq 0.99.$$

因为 σ 与 r 独立，取联合期望并交换取期望的顺序，

$$\mathbb{E}_{\sigma,r}[I(\sigma, r)] \geq 0.99, \qquad \mathbb{E}_\sigma\big[\mathbb{P}_r(\mathrm{Time}(A(\sigma, r)) \geq 0.5n \log n)\big] \geq 0.99.$$

令

$$f_\sigma = \mathbb{P}_r[\mathrm{Time}(A(\sigma, r)) \geq 0.5n \log n], \qquad f_\sigma \in [0, 1].$$

则 $\mathbb{E}_\sigma[f_\sigma] \geq 0.99$，即 $\mathbb{E}_\sigma[1 - f_\sigma] \leq 0.01$。根据 Markov 不等式，

$$\mathbb{P}_\sigma[1 - f_\sigma > 0.2] \leq \frac{0.01}{0.2} = 0.05, \qquad \mathbb{P}_\sigma[1 - f_\sigma \leq 0.2] \geq 0.95.$$

因此至少 95% 的排列满足 $f_\sigma \geq 0.8$。对这些排列，利用运行时间的非负性，

$$\begin{aligned} \mathbb{E}_r[\mathrm{Time}(A(\sigma, r))] &\geq \mathbb{E}_r[I(\sigma, r)] \cdot 0.5n \log n \\ &\geq 0.8 \cdot 0.5n \log n = 0.4n \log n. \end{aligned}$$

这比所要求的 90% 更强。

问题 4（20 分）

(a) 提供算法 A，保证输入 σ 为 $\{1, \ldots, n\}$ 的随机排列时，A 的平均时间为 $T(n)$，即

$$\mathbb{E}_\sigma[\mathrm{Time}(A(\sigma))] = T(n).$$

设计随机算法 $B(\sigma, r)$ 调用 A，使对任意 σ，B 的期望时间为 $O(T(n))$：

$$\forall \sigma, \quad \mathbb{E}_r[\mathrm{Time}(B(\sigma, r))] = O(T(n)).$$

(b) 假设 n 是奇数，算法 A 在 $T(n)$ 时间内找到输入 n 个数的中位数（第 $(n+1)/2$ 大的数）。设计算法 B：对输入的 n 个数和参数 k，在 $T(O(n))$ 时间内找到第 k 大的数。对照问题 2（c），这表明寻找第 k 大的数在 k 为中位秩时最难。

Solution.

(a)

Algorithm 2 SortWithA

```
1: procedure SortWithA(σ, r)
2:     for i = 1 to n do
3:         x ← rand_r(i, n)
4:         交换 σ_i 与 σ_x
5:     end for
6:     return AlgorithmA(σ)
7: end procedure
```

其中 $\mathrm{rand}_r(x, y)$ 表示使用随机种子 r，生成 $[x, y]$ 中的均匀随机整数。第 2–5 行将原排列 σ 更新为一个新排列 σ'，并不改变待排序的元素集合，因此返回的结果正确。

Lemma 2.

循环结束时，新的排列 σ' 是均匀随机排列。

证明. 第 i 步等价于从第 i 至 n 个位置中任意选择一个元素，与 σ_i 交换，随后处理第 $i+1$ 至 n 个位置的子问题。使用数学归纳法证明所有排列的生成概率相同。

当 $i = n$ 时，只有一个元素，结论显然成立。假设从 $i+1$ 开始时结论成立，考虑从 i 开始的情况：任意剩余元素都有 $1/(n-i+1)$ 的概率成为 σ'_i；对每一种首元素选择，其余元素由归纳假设等概率排列。因此任意完整后缀排列的概率为

$$\frac{1}{n-i+1} \cdot \frac{1}{(n-i)!} = \frac{1}{(n-i+1)!}.$$

由归纳法得到从 1 至 n 的随机性，即整个排列均匀随机。 □

根据引理，对任意原排列 σ，

$$\mathbb{E}_r[\mathrm{Time}(B(\sigma, r))] = O(n) + T(n).$$

排序算法的 $T(n) = \Omega(n)$，因此 $O(n) = O(T(n))$，得到

$$\forall \sigma, \quad \mathbb{E}_r[\mathrm{Time}(B(\sigma, r))] = O(T(n)).$$

(b) 将第 k 大转换为第 $q = n - k + 1$ 小。通过添加 $+\infty$ 或 $-\infty$，使所求元素成为新数组的中位数。

Algorithm 3 FindByMedian

```
1: procedure FindByMedian(A_{1...n}, k)
2:     q ← n − k + 1
3:     if q > n/2 then
4:         for i = n + 1 to 2q − 1 do
5:             A_i ← +∞
6:         end for
7:         return AlgorithmA(A_{1...2q−1})
8:     else
9:         for i = n + 1 to 2n + 1 − 2q do
10:            A_i ← −∞
11:        end for
12:        return AlgorithmA(A_{1...2n+1−2q})
13:    end if
14: end procedure
```

证明正确性如下。

- 当 $q > n/2$ 时，$2q-1 > n-1$，所以 $2q-1 \geq n$。添加 $+\infty$ 直到共有 $2q-1$ 个元素，此时中位数是第
$$\frac{(2q-1)+1}{2} = q$$
小的元素，正好是原数组的第 q 小，即第 k 大。
- 当 $q \leq n/2$ 时，$2n+1-2q \geq n+1 > n$。添加 $-\infty$ 直到共有 $2n+1-2q$ 个元素。新数组中位数的升序秩为
$$\frac{(2n+1-2q)+1}{2} = n+1-q.$$
减去新增的 $n+1-2q$ 个 $-\infty$，它在原数组中的升序秩为
$$(n+1-q)-(n+1-2q) = q,$$
因而仍是原数组第 k 大的元素。

综上，算法正确。两种情况下传给 AlgorithmA 的元素数均为奇数，并且

$$2q-1 \leq 2n, \qquad 2n+1-2q \leq 2n.$$

因此调用 AlgorithmA 的元素数量为 $O(n)$。在 $T(n) = \Omega(n)$ 的假设下，添加元素需要 $O(n)$ 时间，总时间复杂度为

$$T(O(n)) + O(n) = T(O(n)).$$